

1 Introduction

NP-hard optimization problems exhibit a rich set of possibilities, all the way from allowing approximability to any required degree, to essentially not allowing approximability at all. Despite this diversity, underlying the process of design of approximation algorithms are some common principles. We will explore these in the current chapter.

An optimization problem is polynomial time solvable only if it has the algorithmically relevant combinatorial structure that can be used as “footholds” to efficiently home in on an optimal solution. The process of designing an exact polynomial time algorithm is a two-pronged attack: unraveling this structure in the problem and finding algorithmic techniques that can exploit this structure.

Although **NP**-hard optimization problems do not offer footholds for finding optimal solutions efficiently, they may still offer footholds for finding near-optimal solutions efficiently. So, at a high level, the process of design of approximation algorithms is not very different from that of design of exact algorithms. It still involves unraveling the relevant structure and finding algorithmic techniques to exploit it. Typically, the structure turns out to be more elaborate, and often the algorithmic techniques result from generalizing and extending some of the powerful algorithmic tools developed in the study of exact algorithms.

On the other hand, looking at the process of designing approximation algorithms a little more closely, one can see that it has its own general principles. We illustrate some of these principles in Section 1.1, using the following simple setting.

Problem 1.1 (Vertex cover) Given an undirected graph $G = (V, E)$, and a cost function on vertices $c : V \rightarrow \mathbf{Q}^+$, find a minimum cost *vertex cover*, i.e., a set $V' \subseteq V$ such that every edge has at least one endpoint incident at V' . The special case, in which all vertices are of unit cost, will be called the *cardinality vertex cover problem*.

Since the design of an approximation algorithm involves delicately attacking **NP**-hardness and salvaging from it an efficient approximate solution, it will be useful for the reader to review some key concepts from complexity theory. Appendix A and some exercises in Section 1.3 have been provided for this purpose.

It is important to give precise definitions of an **NP**-optimization problem and an approximation algorithm for it (e.g., see Exercises 1.9 and 1.10). Since these definitions are quite technical, we have moved them to Appendix A. We provide essentials below to quickly get started.

An **NP**-optimization problem Π is either a minimization or a maximization problem. Each valid instance I of Π comes with a nonempty set of feasible solutions, each of which is assigned a nonnegative rational number called its objective function value. There exist polynomial time algorithms for determining validity, feasibility, and the objective function value. A feasible solution that achieves the optimal objective function value is called an optimal solution. $\text{OPT}_\Pi(I)$ will denote the objective function value of an optimal solution to instance I . We will shorten this to OPT when there is no ambiguity. For the problems studied in this book, computing $\text{OPT}_\Pi(I)$ is **NP-hard**.

For example, valid instances of the vertex cover problem consist of an undirected graph $G = (V, E)$ and a cost function on vertices. A feasible solution is a set $S \subseteq V$ that is a cover for G . Its objective function value is the sum of costs of all vertices in S . A minimum cost such set is an optimal solution.

An approximation algorithm, $\mathcal{A}$, for Π produces, in polynomial time, a feasible solution whose objective function value is “close” to the optimal; by “close” we mean within a guaranteed factor of the optimal. In the next section, we will present a factor 2 approximation algorithm for the cardinality vertex cover problem, i.e., an algorithm that finds a cover of cost $\leq 2 \cdot \text{OPT}$ in time polynomial in $|V|$.

1.1 Lower bounding OPT

When designing an approximation algorithm for an **NP-hard** **NP**-optimization problem, one is immediately faced with the following dilemma. In order to establish the approximation guarantee, the cost of the solution produced by the algorithm needs to be compared with the cost of an optimal solution. However, for such problems, not only is it **NP-hard** to find an optimal solution, but it is also **NP-hard** to compute the cost of an optimal solution (see Appendix A). In fact, in Section A.5 we show that computing the cost of an optimal solution (or even solving its decision version) is precisely the difficult core of such problems. So, how do we establish the approximation guarantee? Interestingly enough, the answer to this question provides a key step in the design of approximation algorithms.

Let us demonstrate this in the context of the cardinality vertex cover problem. We will get around the difficulty mentioned above by coming up with a “good” polynomial time computable *lower bound* on the size of the optimal cover.

1.1.1 An approximation algorithm for cardinality vertex cover

We provide some definitions first. Given a graph $H = (U, F)$, a subset of the edges $M \subseteq F$ is said to be a *matching* if no two edges of M share an endpoint. A matching of maximum cardinality in H is called a *maximum matching*, and a matching that is maximal under inclusion is called a *maximal matching*. A maximal matching can clearly be computed in polynomial time by simply greedily picking edges and removing endpoints of picked edges. More sophisticated means lead to polynomial time algorithms for finding a maximum matching as well.

Let us observe that the size of a maximal matching in G provides a lower bound. This is so because *any* vertex cover has to pick at least one endpoint of each matched edge. This lower bounding scheme immediately suggests the following simple algorithm:

Algorithm 1.2 (Cardinality vertex cover)

Find a maximal matching in G and output the set of matched vertices.

Theorem 1.3 *Algorithm 1.2 is a factor 2 approximation algorithm for the cardinality vertex cover problem.*

Proof: No edge can be left uncovered by the set of vertices picked – otherwise such an edge could have been added to the matching, contradicting its maximality. Let M be the matching picked. As argued above, $|M| \leq \text{OPT}$. The approximation factor follows from the observation that the cover picked by the algorithm has cardinality $2|M|$, which is at most $2 \cdot \text{OPT}$. $\square$

Observe that the approximation algorithm for vertex cover was very much related to, and followed naturally from, the lower bounding scheme. This is in fact typical in the design of approximation algorithms. In Part II of this book, we show how linear programming provides a unified way of obtaining lower bounds for several fundamental problems. The algorithm itself is designed around the LP that provides the lower bound.

1.1.2 Can the approximation guarantee be improved?

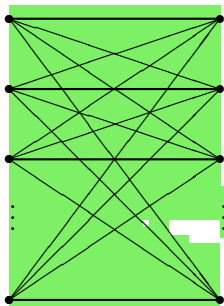
The following questions arise in the context of improving the approximation guarantee for cardinality vertex cover:

1. Can the approximation guarantee of Algorithm 1.2 be improved by a better analysis?

2. Can an approximation algorithm with a better guarantee be designed using the lower bounding scheme of Algorithm 1.2, i.e., size of a maximal matching in G ?
3. Is there some other lower bounding method that can lead to an improved approximation guarantee for vertex cover?

Example 1.4 shows that the answer to the first question is “no”, i.e., the analysis presented above for Algorithm 1.2 is tight. It gives an infinite family of instances in which the solution produced by Algorithm 1.2 is twice the optimal. An infinite family of instances of this kind, showing that the analysis of an approximation algorithm is tight, will be referred to as a *tight example*. The importance of finding tight examples for an approximation algorithm one has designed cannot be overemphasized. They give critical insight into the functioning of the algorithm and have often led to ideas for obtaining algorithms with improved guarantees. (The reader is advised to run algorithms on the tight examples presented in this book.)

Example 1.4 Consider the infinite family of instances given by the complete bipartite graphs $K_{n,n}$.



When run on $K_{n,n}$, Algorithm 1.2 will pick all $2n$ vertices, whereas picking one side of the bipartition gives a cover of size n . $\square$

Let us assume that we will establish the approximation factor for an algorithm by simply comparing the cost of the solution it finds with the lower bound. Indeed, almost all known approximation algorithms operate in this manner. Under this assumption, the answer to the second question is also “no”. This is established in Example 1.5, which gives an infinite family of instances on which the lower bound, of size of a maximal matching, is in fact half the size of an optimal vertex cover. In the case of linear-programming-based approximation algorithms, the analogous question will be answered by determining a fundamental quantity associated with the linear programming relaxation – its integrality gap (see Chapter 12).

The third question, of improving the approximation guarantee for vertex cover, is currently a central open problem in the field of approximation algorithms (see Section 30.1).

Example 1.5 The lower bound, of size of a maximal matching, is half the size of an optimal vertex cover for the following infinite family of instances. Consider the complete graph K_n , where n is odd. The size of any maximal matching is $(n - 1)/2$, whereas the size of an optimal cover is $n - 1$. $\square$

1.2 Well-characterized problems and min–max relations

Consider decision versions of the cardinality vertex cover and maximum matching problems.

- Is the size of the minimum vertex cover in G at most k ?
- Is the size of the maximum matching in G at least l ?

Both these decision problems are in **NP** and therefore have Yes certificates (see Appendix A for definitions). Do these problems also have No certificates? We have already observed that the size of a maximum matching is a lower bound on the size of a minimum vertex cover. If G is bipartite, then in fact equality holds; this is the classic König-Egerváry theorem.

Theorem 1.6 *In any bipartite graph,*

$$\max_{\text{matching } M} |M| = \min_{\text{vertex cover } U} |U|.$$

Therefore, if the answer to the first decision problem is “no”, there must be a matching of size $k + 1$ in G that suffices as a certificate. Similarly, a vertex cover of size $l - 1$ must exist in G if the answer to the second decision problem is “no”. Hence, when restricted to bipartite graphs, both vertex cover and maximum matching problems have No certificates and are in **co-NP**. In fact, both problems are also in **P** under this restriction. It is easy to see that any problem in **P** trivially has Yes as well as No certificates (the empty string suffices). This is equivalent to the statement that $\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{co-NP}$. It is widely believed that the containment is strict; the conjectured status of these classes is depicted below.

